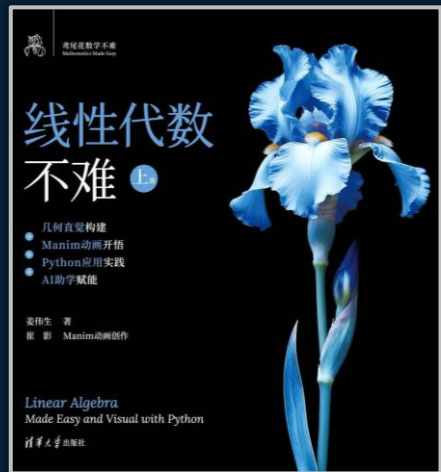


08 几何变换

8.3 旋转

旋转矩阵：正交矩阵、行列式为1、连续旋转的交换律，从平面到三维空间绕轴旋转



鸢尾花书·《线性代数不难》 | 开源代码与书稿：<https://github.com/Visualize-ML>

本节你将掌握的核心技能

1

矩阵乘法实现绕原点平面旋转： $y=R(\theta)x$ ， θ 为逆时针旋转角度

2

旋转矩阵 $R(\theta)$ 为正交矩阵： $R^{-1}=R^T$ ， $R @ R^T = R^T @ R = I$ ，行列式恒为1

3

格拉姆矩阵 $R^T R$ 与列向量内积、L2范数的关系

4

连续平面旋转满足交换律：
 $R(\beta)R(\alpha) = R(\alpha)R(\beta) = R(\alpha+\beta)$

5

交换旋转、缩放的次序会影响结果，矩阵乘法一般不满足交换律

6

矩阵乘法实现三维绕轴旋转：绕 x_1 、 x_2 、 x_3 轴分别独立旋转，遵循右手定则

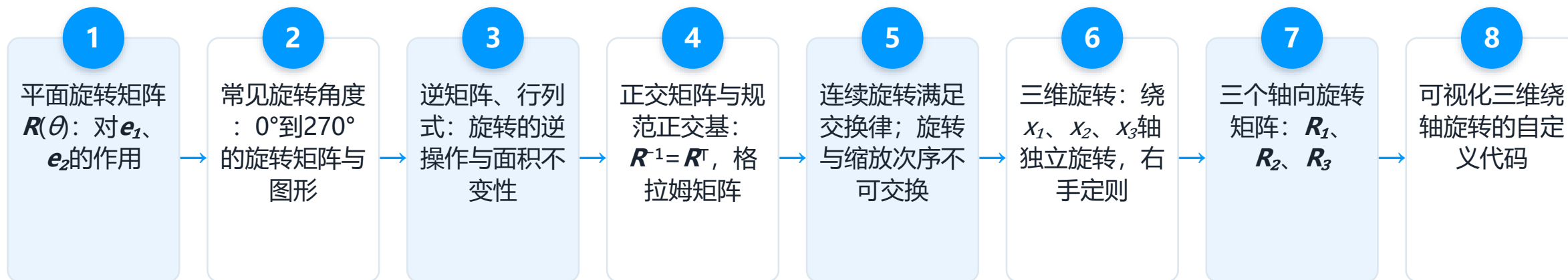
7

三维旋转矩阵同样是正交矩阵，行列式为1，可用拉普拉斯展开验证

8

三维旋转，绕不同轴的旋转次序不能交换；只有绕同一轴旋转才能交换顺序

从平面旋转到三维绕轴旋转



本章每个几何变换，建议从这几方面看

- ① 矩阵乘法几何视角：矩阵对零向量 $\mathbf{0}$ 、单位向量 $\mathbf{e}_1$ 、 $\mathbf{e}_2$ 、全1列向量 $\mathbf{1}$ 的变换
- ② 看行列式：绝对值是否大于1；正负还是零；特殊矩阵（对角、上/下三角）的行列式
- ③ 从行列式看几何变换导致的面积或体积变换
- ④ 从逆矩阵（先看是否存在）看逆变换；特殊矩阵的逆矩阵
- ⑤ 线性组合角度看平行四边形网格、平行六面体形状变换，是否发生降维
- ⑥ 从基底角度看变换矩阵是否构成基底，以及构成怎样的基底
- ⑦ 变换矩阵的特点：是否对称、对角、正交、可逆，矩阵乘法是否满足交换律

绕原点逆时针旋转角度 θ 对应的变换矩阵 $R(\theta)$

$$R(\theta) = [\cos\theta, -\sin\theta; \sin\theta, \cos\theta]$$

- 旋转变换在机器学习、计算机图形学、机器人、物理学中都有广泛应用，本书后文谱分解、主成分分析、奇异值分解都会用到旋转
- 旋转通常用矩阵乘法完成；本节先讲平面旋转，再讲三维旋转

列向量 $\mathbf{x}$ 绕原点逆时针旋转 θ 后的新坐标 $\mathbf{y}$

$$[y_1, y_2]^T = R(\theta)[x_1, x_2]^T$$

- 在平面上，给定一个列向量 $\mathbf{x}$ ，绕原点逆时针旋转角度 θ 后得到新向量 $\mathbf{y}$
- 旋转变换不改变向量长度，只改变向量方向——这是旋转区别于缩放的关键特征

图1 • R 对 e_1 、 e_2 的作用

- $Re_1 = [\cos \theta, \sin \theta]^T$, 相当于提取 R 的第一列; $Re_2 = [-\sin \theta, \cos \theta]^T$, 相当于提取 R 的第二列

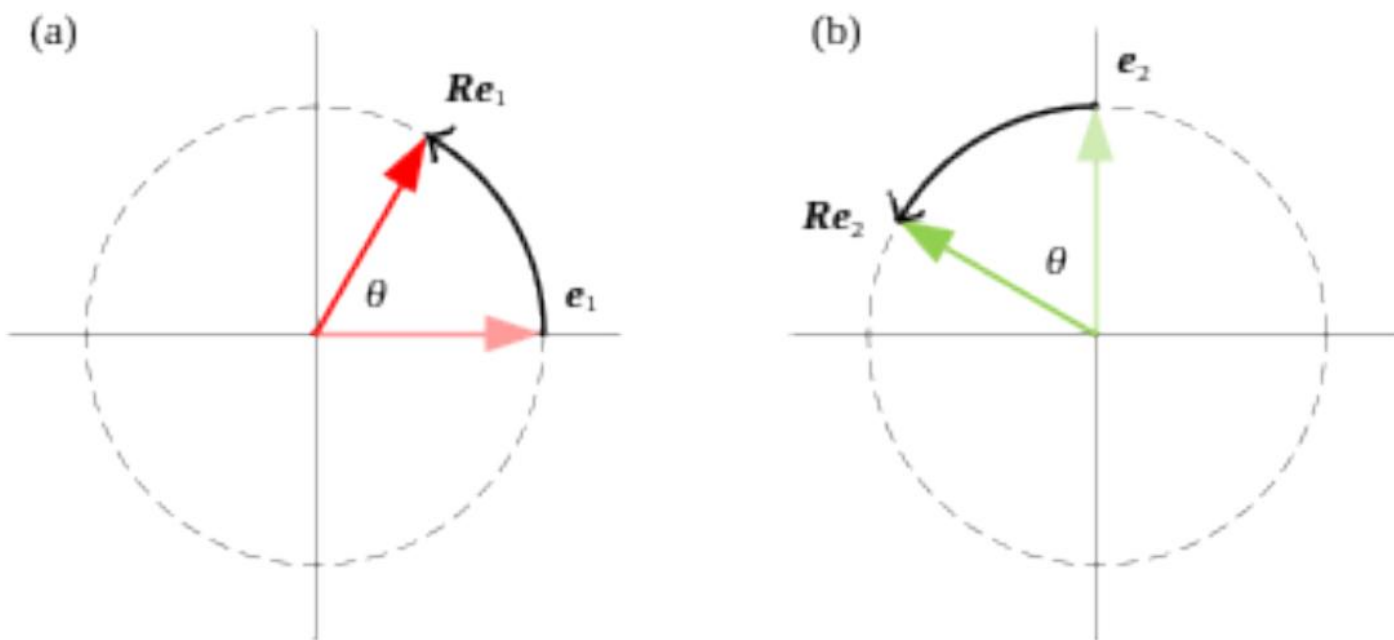


图 1. R 对 e_1 、 e_2 的作用

Re_1 与 Re_2 分别是 e_1 、 e_2 绕原点旋转 θ 后的新位置

$$Re_1 = [\cos\theta, \sin\theta]^T \quad Re_2 = [-\sin\theta, \cos\theta]^T$$

- 如图1(a)所示, $[\cos\theta, \sin\theta]^T$ 是 e_1 绕原点逆时针旋转 θ 的新位置
- 如图1(b)所示, $[-\sin\theta, \cos\theta]^T$ 是 e_2 绕原点逆时针旋转 θ 的新位置

图2 • 旋转的两个例子

- (a) 旋转角度为 $2\pi/3(120^\circ)$; (b) 旋转角度为 $-3\pi/4(-135^\circ)$, 即顺时针 135°)

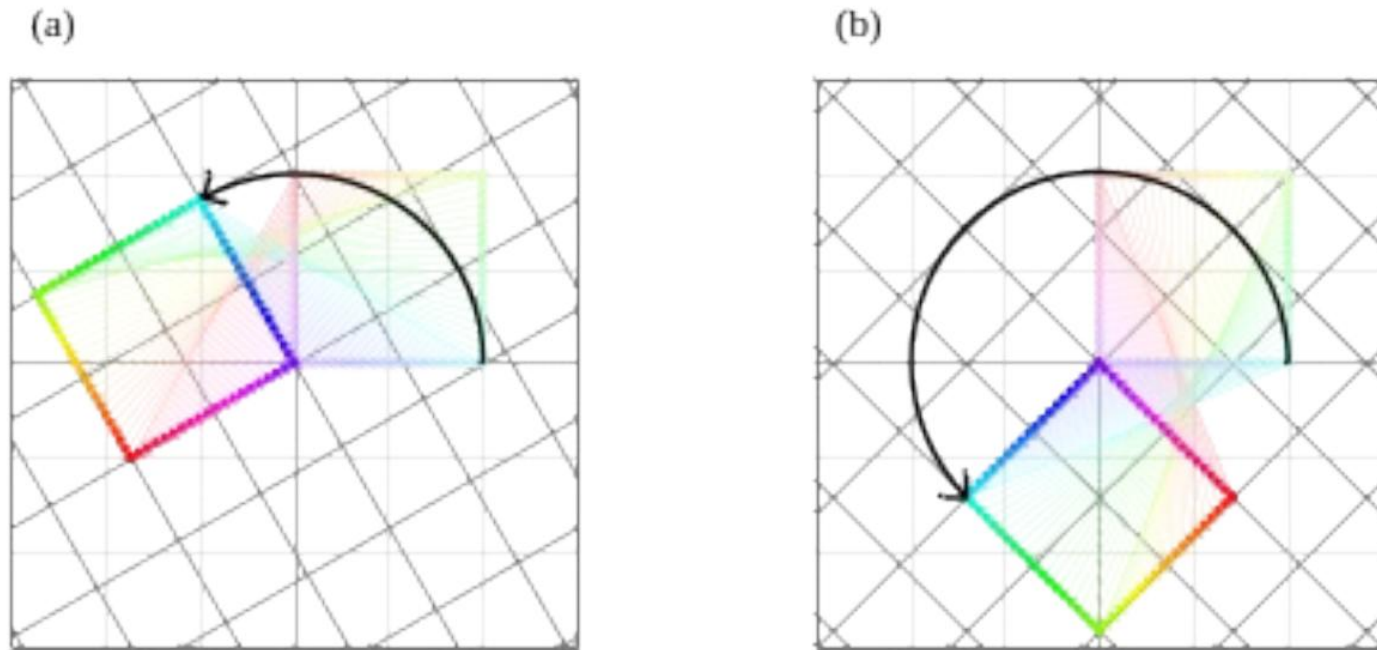
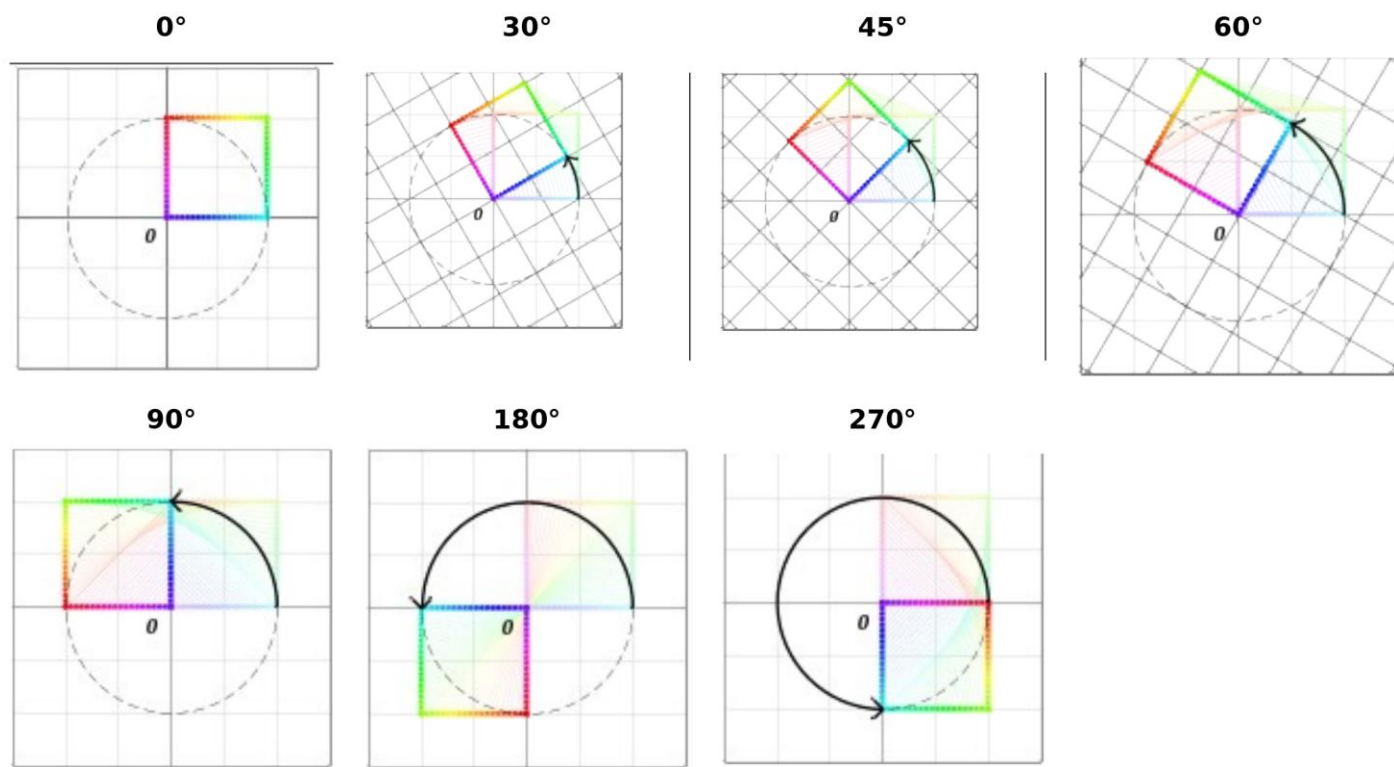


图 2. 旋转的两个例子

表1 • 几个常见的旋转角度

- 0° 相当于单位矩阵 I , 不发生变化; 90° 、 180° 、 270° 的旋转矩阵元素只含 0、1、 -1 , 计算简便



顺时针旋转 θ 对应逆时针旋转 $-\theta$

$$R^{-1}(\theta) = R(-\theta) = [\cos\theta, \sin\theta; -\sin\theta, \cos\theta]$$

- 对旋转矩阵 $R(\theta)$ 求逆，得到的矩阵实际上表示绕原点顺时针旋转 θ 度
- 这也提示我们：旋转的逆操作仍然是一次旋转，只是方向相反

图3 • 旋转的逆运算

- 先用 R 逆时针旋转，再用 R^{-1} 顺时针旋转相同角度，图形回到原始位置

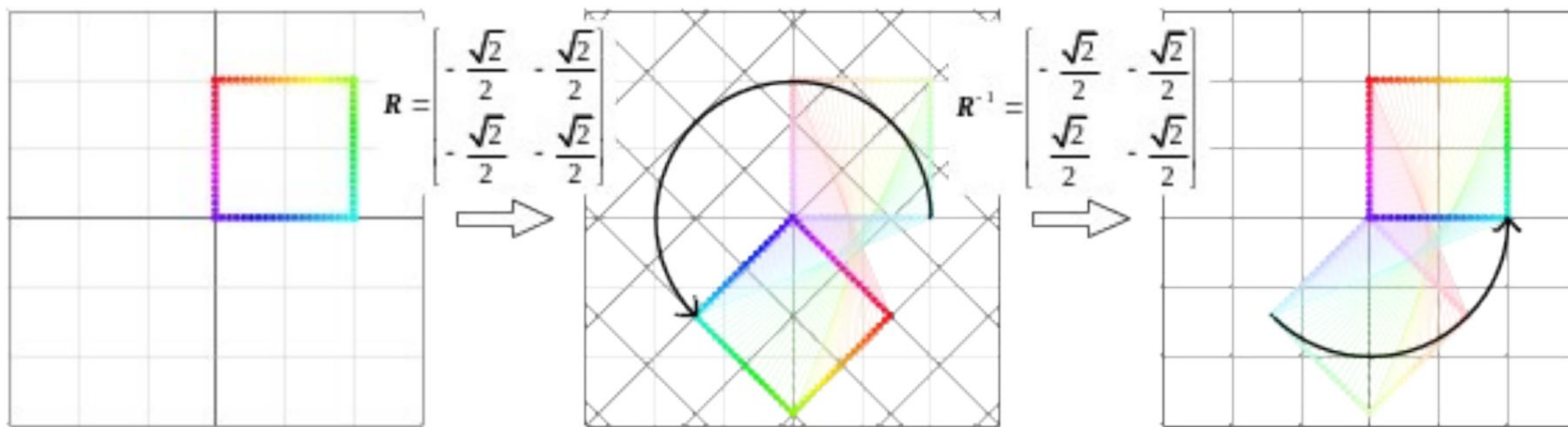


图 3. 旋转的逆运算

二维旋转矩阵的行列式恒为1

$$\det(R(\theta)) = \cos^2\theta + \sin^2\theta = 1$$

- 这说明平面旋转不改变几何形状的面积；这个(绕原点)旋转也不改变向量先后顺序
- 行列式恒为1，是旋转区别于缩放、镜像等其他几何变换的重要标志

旋转矩阵的逆 R^{-1} 等于它的转置矩阵

$$R^{-1} = R^T \quad R @ R^T = R^T @ R = I$$

- 比较 $R^{-1}(\theta)$ 与 $R(\theta)$ 的转置，我们发现两者完全相同
- 这意味着 R 为正交矩阵(orthogonal matrix)——正交矩阵的逆等于其转置

$R=[r_1, r_2]$ 的列向量构成规范正交基

$$\|r_1\|_2^2 = r_1^T @ r_1 = 1 \quad \|r_2\|_2^2 = r_2^T @ r_2 = 1$$

- R 的每一列都是单位向量，L2范数的平方为1；此外两列向量正交，即 $r_2^T @ r_1 = r_1^T @ r_2 = 0$
- 这意味着 $R=[r_1, r_2]$ 为规范正交基；特别地，当 θ 为0时， $R=I$ ，得到标准正交基

R 的转置乘 R , 叫做 R 的格拉姆矩阵(Gram matrix)

$$R^T R = [r_1^T @ r_1, r_1^T @ r_2; r_2^T @ r_1, r_2^T @ r_2] = I$$

- 格拉姆矩阵对角线上是列向量自身的内积, 等价列向量L2范数的平方; 非对角线元素对应成对向量内积
- 本书后文将专门讲解格拉姆矩阵——它是理解向量组正交性、独立性的重要工具

先旋转 α 、后旋转 β ，总旋转矩阵为 $R(\alpha + \beta)$

$$R(\beta)R(\alpha) = R(\alpha)R(\beta) = R(\alpha + \beta)$$

- 本书前文提到矩阵乘法一般不满足交换律，但连续两次绕原点旋转是特殊情况：可以合并成一次旋转
- 这说明平面绕原点旋转变换是可累加的——多次旋转等价于单次旋转对应角度的累加

图4 • 两个 2×2 旋转矩阵连乘满足交换律

- 上排先 R_1 后 R_2 ，下排先 R_2 后 R_1 ——两种次序得到完全相同的最终结果

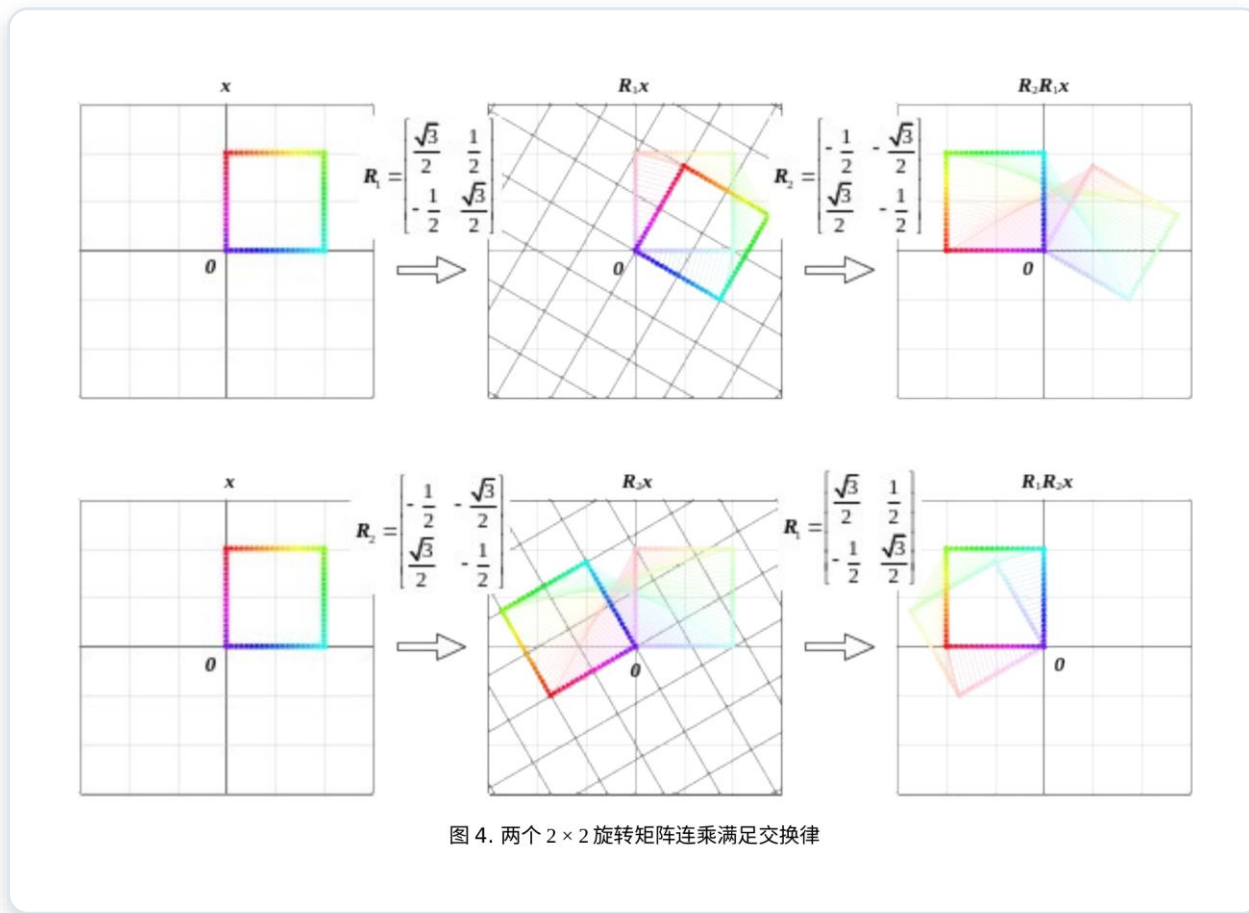


图 4. 两个 2×2 旋转矩阵连乘满足交换律

先缩放、再旋转 $\neq$ 先旋转、再缩放

$$RS \neq SR$$

- 本书前文提到过矩阵乘法一般不满足交换律，即 $AB \neq BA$ ；旋转与缩放的复合正是典型例子
- 反复强调：对于矩阵乘法 $ABCx$ ，对列向量 x 的作用顺序从左向右，即 C 、 B 、 A

图5 • 先缩放，再旋转

- $S = \text{diag}(1/2, 2)$ 先把正方形压扁拉伸， R 再将结果旋转 60° ，得到倾斜的平行四边形

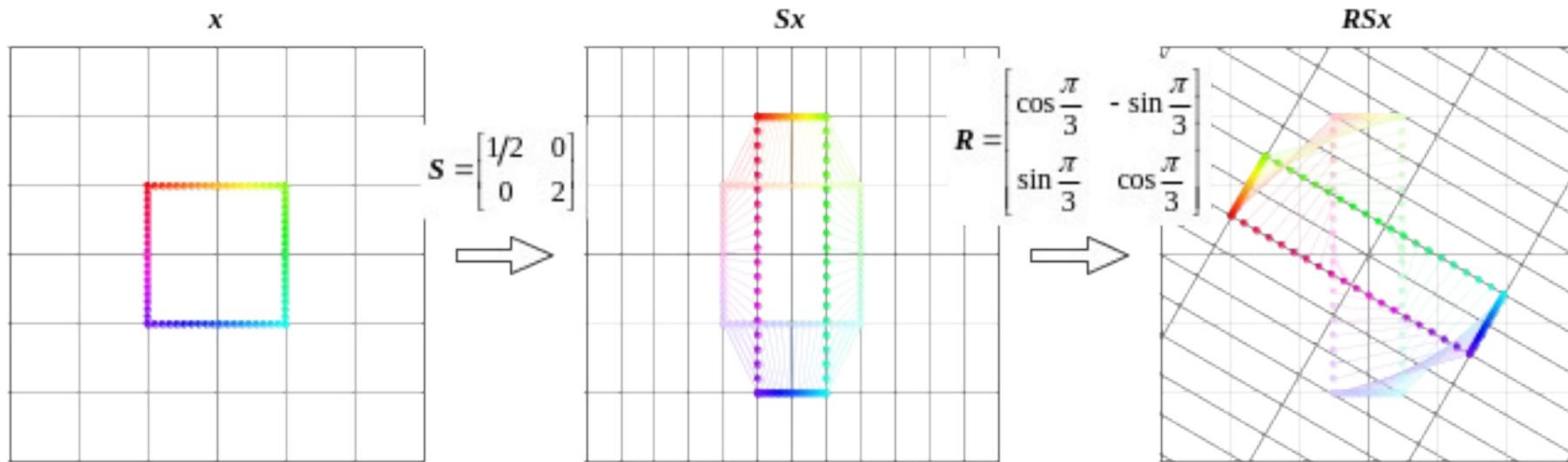


图 5. 先缩放，再旋转

图6 • 先旋转，再缩放

- R 先将正方形旋转 60° ， $S=\text{diag}(1/2, 2)$ 再压扁拉伸——最终形状与图5明显不同

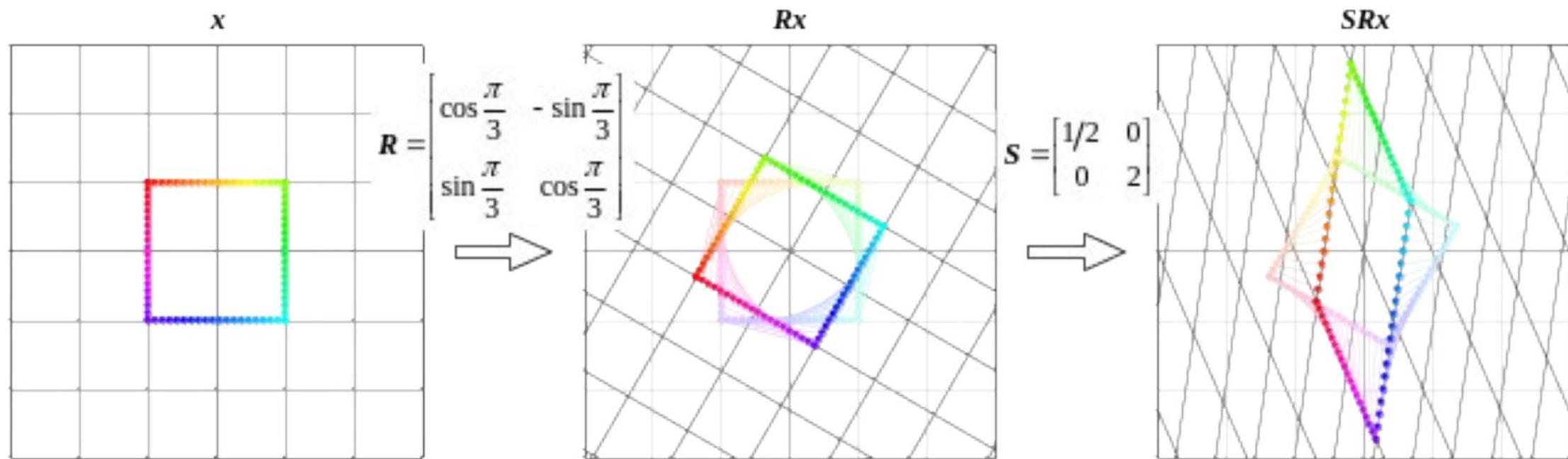


图 6. 先旋转，再缩放

绕 x_1 轴逆时针旋转角度 θ_1 对应的变换矩阵 R_1

$$R_1 = [1, 0, 0; 0, \cos\theta_1, -\sin\theta_1; 0, \sin\theta_1, \cos\theta_1]$$

- 在三维空间中，几何体可以独立绕 x_1 、 x_2 、 x_3 轴分别独立旋转
- 旋转正负方向遵循右手定则：拇指指向旋转轴正方向，其余四指弯曲方向为正向(逆时针)旋转

图7 · 几何体绕三轴独立旋转

- 单位正方体可以分别绕 x_1 、 x_2 、 x_3 轴独立旋转 θ_1 、 θ_2 、 θ_3 角度

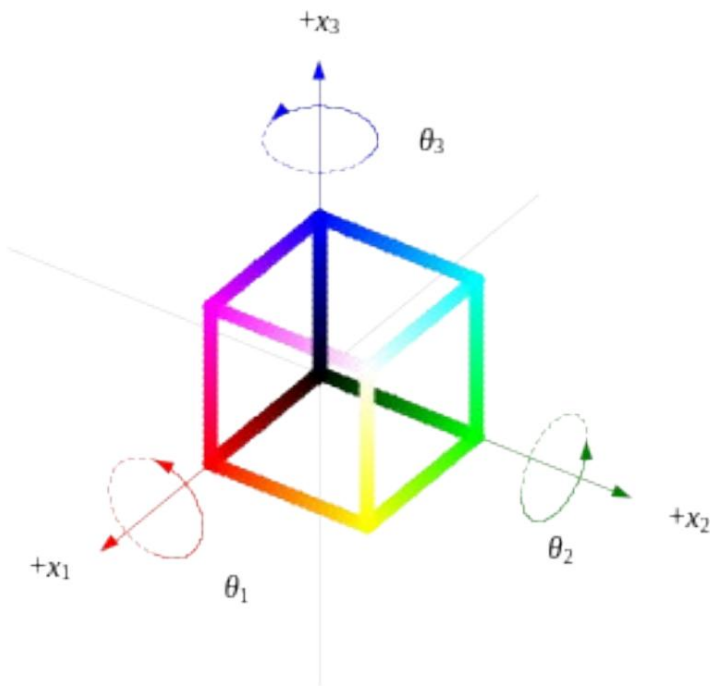


图 7. 几何体绕三轴独立旋转

图8 • 三个旋转角度

- 拇指指向旋转轴正方向，其余四指弯曲方向即为正向(逆时针)旋转方向——绕三个轴的旋转都遵循同一规则

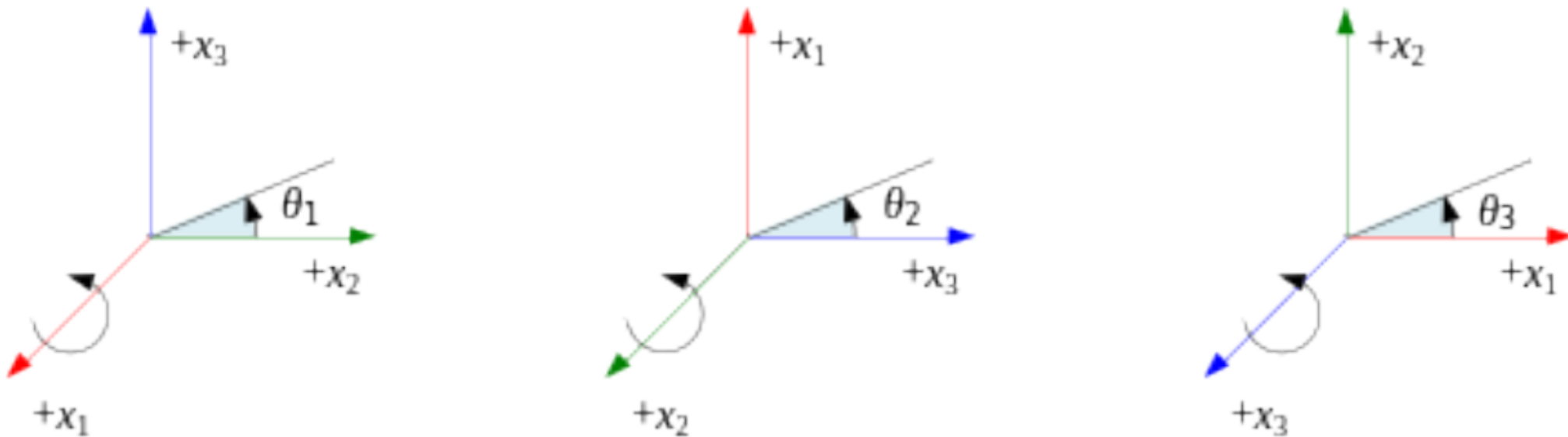


图 8. 三个旋转角度

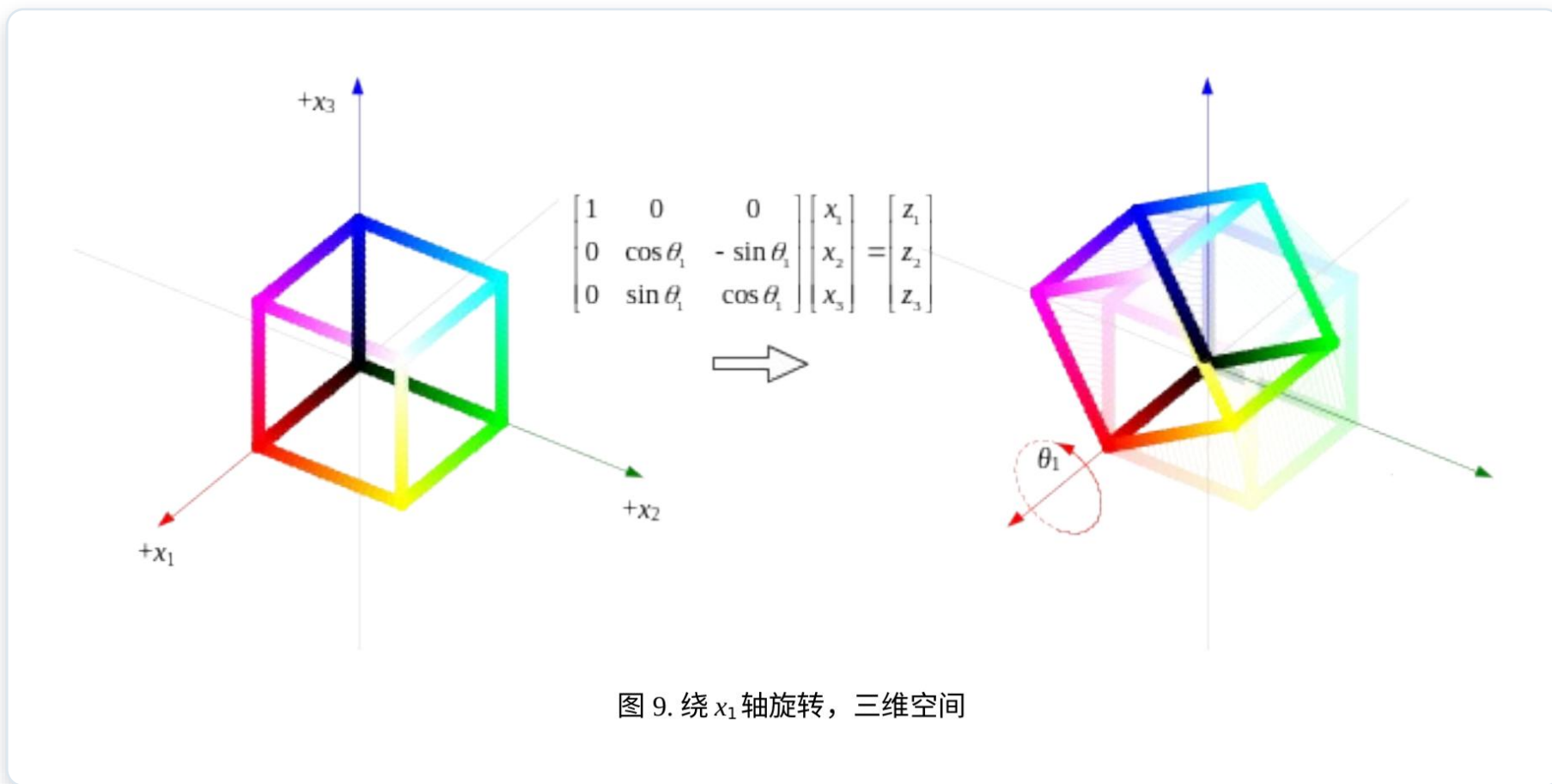
三维列向量 $\mathbf{x}$ 绕 x_1 轴旋转 θ_1 后的坐标 $\mathbf{z}$

$$z_1 = x_1 \quad z_2 = x_2 \cos \theta_1 - x_3 \sin \theta_1 \quad z_3 = x_2 \sin \theta_1 + x_3 \cos \theta_1$$

- $\mathbf{R}_1$ 只改变 x_2 、 x_3 两个坐标， x_1 坐标保持不变——这正是“绕 x_1 轴”旋转的几何含义
- 展开 $\mathbf{R}_1 \mathbf{x} = \mathbf{z}$ ，得到上述三个等式，形式上与二维旋转矩阵完全一致

图9 • 绕 x_1 轴旋转，三维空间

- 单位正方体绕 x_1 轴逆时针旋转 θ_1 后， x_2x_3 平面内的四条棱发生转动， x_1 方向的棱保持不变



R_1 同样满足正交矩阵性质，行列式为1

$$R_1^{-1} = R_1^T \quad \det(R_1) = 1$$

- 计算格拉姆矩阵 $R_1^T R_1 = R_1 R_1^T = I$ ，说明 R_1 也是正交矩阵，在三维空间中为规范正交基
- 用拉普拉斯展开沿第一行计算行列式（因为该行0最多，计算量最小），得到 $\det(R_1) = 1$ ，体积不发生变化

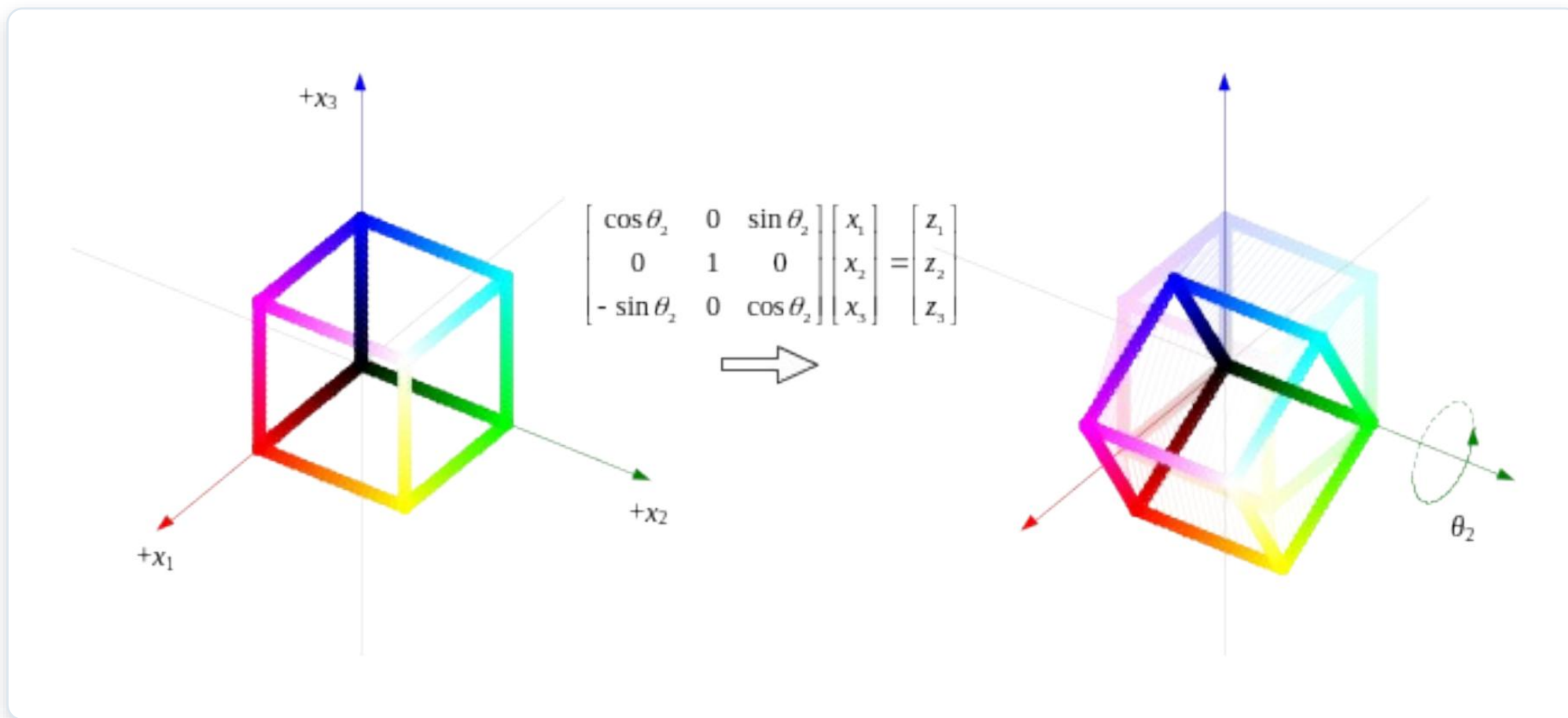
绕 x_2 轴逆时针旋转角度 θ_2 对应的变换矩阵 R_2

$$R_2 = [\cos\theta_2, 0, \sin\theta_2; 0, 1, 0; -\sin\theta_2, 0, \cos\theta_2]$$

- R_2 只改变 x_1 、 x_3 两个坐标， x_2 坐标保持不变
- 同样可验证 R_2 为正交矩阵，行列式为1——请大家自行计算 R_2 的逆矩阵并验证

图10 • 绕 x_2 轴旋转，三维空间

- 单位正方体绕 x_2 轴逆时针旋转 θ_2 后， x_1x_3 平面内的棱发生转动， x_2 方向的棱保持不变



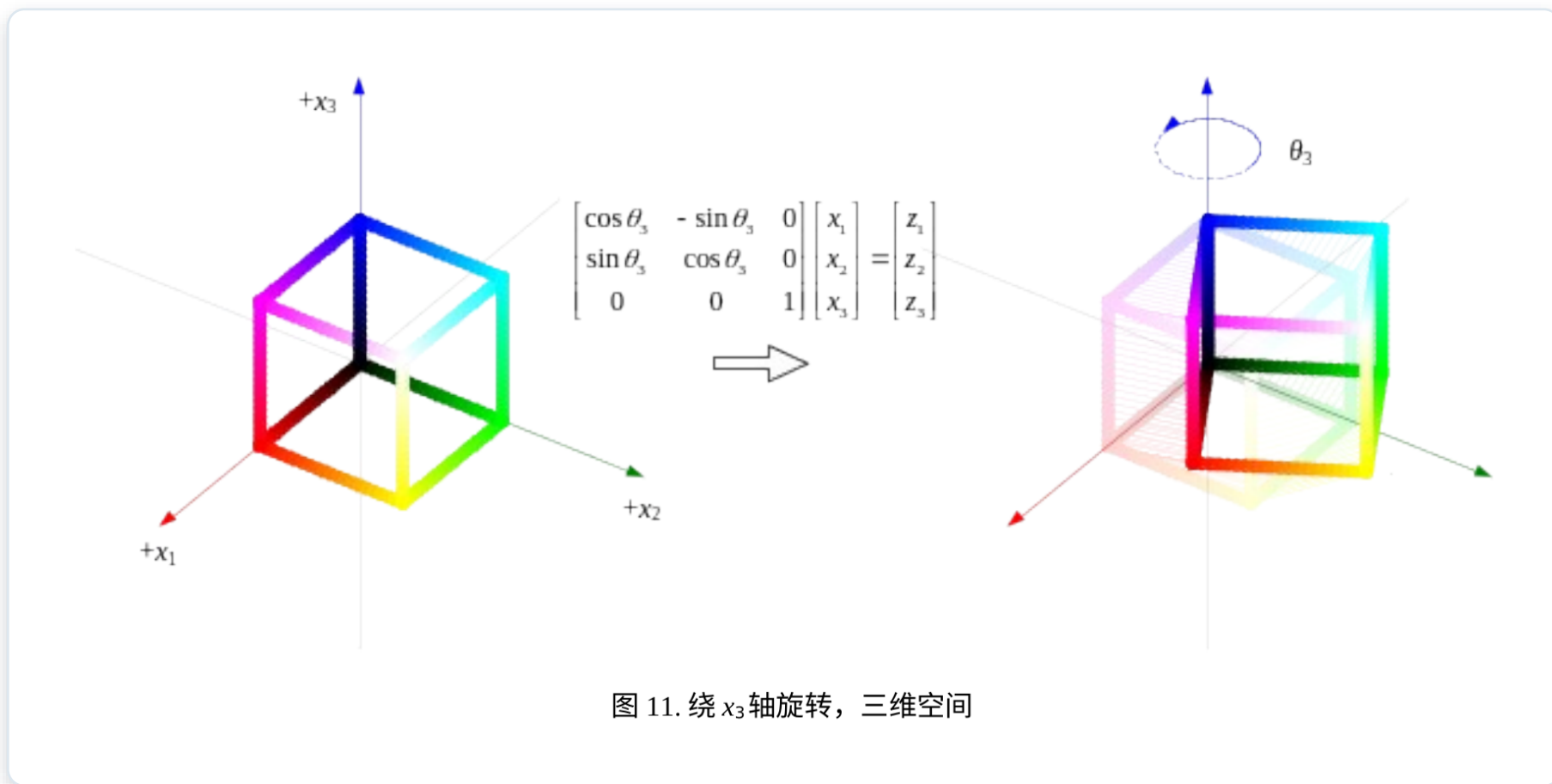
绕 x_3 轴逆时针旋转角度 θ_3 对应的变换矩阵 R_3

$$R_3 = [\cos\theta_3, -\sin\theta_3, 0; \sin\theta_3, \cos\theta_3, 0; 0, 0, 1]$$

- R_3 只改变 x_1 、 x_2 两个坐标， x_3 坐标保持不变，形式上与二维旋转矩阵一致
- 同样可验证 R_3 为正交矩阵，行列式为1——请大家自行计算 R_3 的逆矩阵并验证

图11 • 绕 x_3 轴旋转，三维空间

- 单位正方体绕 x_3 轴逆时针旋转 θ_3 后， x_1x_2 平面内的棱发生转动， x_3 方向的棱保持不变



三维旋转，绕不同轴的旋转次序不能交换

$$R_1 R_2 \neq R_2 R_1 \quad R_1(\theta_\alpha) R_1(\theta_\beta) = R_1(\theta_\beta) R_1(\theta_\alpha)$$

- 绕不同轴的旋转先后顺序不能交换；只有绕同一轴旋转（角度不同）才能交换顺序，这一点类似平面旋转
- 这与平面旋转“任意两次旋转都可交换”的性质不同——三维旋转的复合顺序会实际影响最终姿态

自定义可视化函数：绘制立方体

`plot_cube()`: 绘制原始与变换后立方体

- 把原始立方体顶点数据赋值给colors变量，单位正方体对应RGB空间，每个顶点坐标即颜色色号
- 定义edges列表，列出立方体12条边，每条边由两个顶点索引构成，共8个点组合成12条边
- 用for循环画立方体原始状态所有边（灰色虚线）与变换后所有边（黑色实线）
- 用scatter()分别绘制原始立方体、变换后立方体的顶点，每个顶点设置不同颜色

```
## 可视化函数
def plot_cube(ax, original_cube, transformed_cube):
    colors = original_cube

    # 定义每个卦限的框线顶点对
    edges = [[0, 1], [1, 2], [2, 3], [3, 0],
             [0, 4], [1, 5], [2, 6], [3, 7],
             [4, 5], [5, 6], [6, 7], [7, 4]]

    # 绘制原始立方体框线
    for edge in edges:
        ax.plot(*zip(*original_cube[edge]),
                color="gray", linestyle="dotted")

    # 绘制变换后的立方体框线
    for edge in edges:
        ax.plot(*zip(*transformed_cube[edge]),
                color="black", linestyle="-")

    # 绘制顶点
    ax.scatter(original_cube[:,0], original_cube[:,1],
              original_cube[:,2],
              facecolor=colors, s=50, marker='o', edgecolor='k')

    ax.scatter(transformed_cube[:,0], transformed_cube[:,1],
              transformed_cube[:,2],
              facecolor=colors, s=50, marker='o', edgecolor='k')

    ax.set_xlim(-2, 2); ax.set_ylim(-2, 2); ax.set_zlim(-2, 2)
    ax.axis(False); ax.set_box_aspect([1, 1, 1])
    ax.view_init(elev=30, azim=30)
    ax.set_proj_type('ortho')
    ax.plot((-2,2),(0,0),(0,0),color = 'k')
    ax.plot((0,0),(-2,2),(0,0),color = 'k')
    ax.plot((0,0),(0,0),(-2,2),color = 'k')
```

三维绕轴旋转案例

分别绕 x_1 、 x_2 、 x_3 轴旋转单位立方体

- 定义单位立方体的8个顶点坐标;
 $\theta_1 = \pi/6$, 即30度
- 构造 R_{x1} 旋转矩阵, 用 $\text{cube} @ R_{x1.T}$ 完成绕 x_1 轴的旋转变换, 再调用 $\text{plot_cube}()$ 可视化
- 同样的方式构造 R_{x2} 、 R_{x3} , 分别完成绕 x_2 、 x_3 轴的旋转, 调用代码1定义的自定义函数绘图
- 三段代码结构完全一致, 只是旋转矩阵和旋转轴不同——体现了函数封装、代码复用的思想

```

a  ## 定义单位立方体的 8 个顶点坐标
cube = np.array([[0, 0, 0], [1, 0, 0], [1, 1, 0], [0, 1, 0],
                 [0, 0, 1], [1, 0, 1], [1, 1, 1], [0, 1, 1]])

## 绕 x1 轴旋转
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(111, projection='3d')

theta_1 = np.pi / 6
b  R_x1 = np.array([[1, 0, 0],
                  [0, np.cos(theta_1), -np.sin(theta_1)],
                  [0, np.sin(theta_1), np.cos(theta_1)]])

rotated_cube = cube @ R_x1.T
plot_cube(ax, cube, rotated_cube)

## 绕 x2 轴旋转
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(111, projection='3d')

theta_2 = np.pi / 6
c  R_x2 = np.array([[np.cos(theta_2), 0, np.sin(theta_2)],
                  [0, 1, 0],
                  [-np.sin(theta_2), 0, np.cos(theta_2)]])

rotated_cube = cube @ R_x2.T
plot_cube(ax, cube, rotated_cube)

## 绕 x3 轴旋转
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(111, projection='3d')

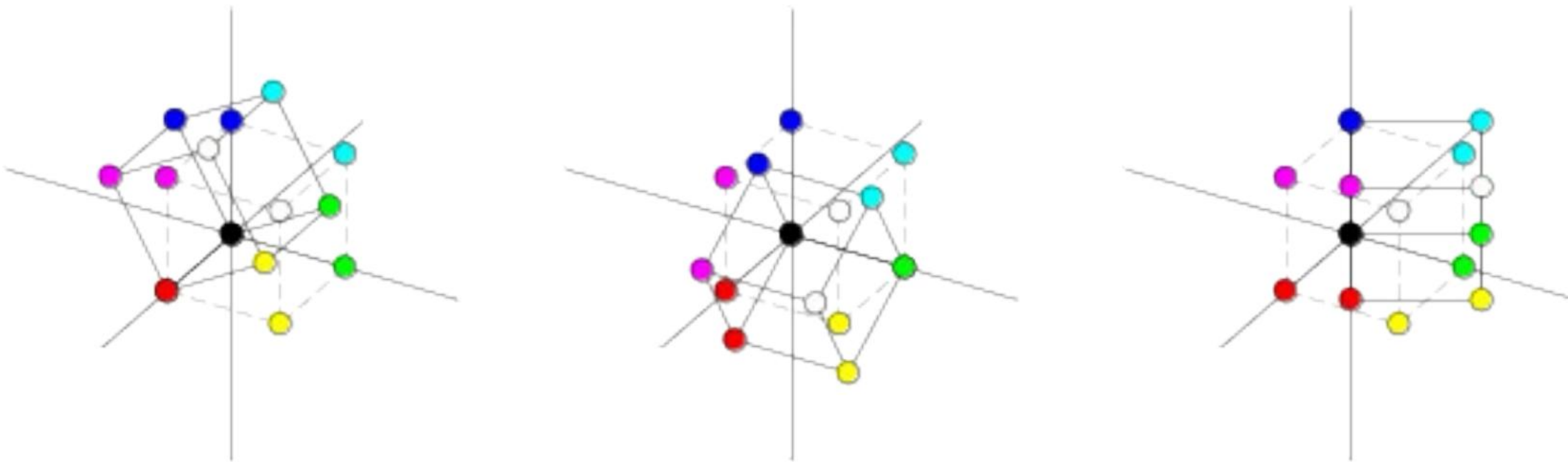
theta_3 = np.pi / 6
d  R_x3 = np.array([[np.cos(theta_3), -np.sin(theta_3), 0],
                  [np.sin(theta_3), np.cos(theta_3), 0],
                  [0, 0, 1]])

rotated_cube = cube @ R_x3.T
plot_cube(ax, cube, rotated_cube)

```

图12 • 三维绕轴旋转案例

- 从左到右依次为绕 x_1 、 x_2 、 x_3 轴旋转30度的结果；灰色虚线为原始立方体，黑色实线为旋转后立方体



从矩阵乘法看旋转变换，观察到了什么？

旋转矩阵 $\mathbf{R}(\theta)$ ：正交矩阵， $\mathbf{R}^{-1} = \mathbf{R}^T$ ，行列式恒为1，不改变面积/体积

连续旋转满足交换律： $\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha + \beta)$ ；但旋转与缩放的次序不可交换

三维旋转：绕 x_1 、 x_2 、 x_3 轴独立旋转，右手定则；同轴旋转可交换，异轴不可交换

格拉姆矩阵 $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ ：列向量单位长度且两两正交，构成规范正交基

本节关键术语

旋转变换 *Rotation Transformation* —— 绕原点或坐标轴转动图形的线性变换

正交矩阵 *Orthogonal Matrix* —— 满足 $\mathbf{R}^{-1} = \mathbf{R}^T$ 的方阵

标准正交基 *Standard Basis* —— $\theta=0$ 时的规范正交基, 即 $\mathbf{I}$ 的列向量

交换律 *Commutative Law* —— 连续旋转满足 $\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\beta)\mathbf{R}(\alpha)$

拉普拉斯展开 *Laplace Expansion* —— 按行/列展开计算行列式的方法

旋转矩阵 *Rotation Matrix* —— $\mathbf{R}(\theta)$, 正交矩阵, 行列式为1

规范正交基 *Orthonormal Basis* —— 列向量单位长度且两两正交

格拉姆矩阵 *Gram Matrix* —— $\mathbf{R}^T \mathbf{R}$, 对角线为L2范数平方, 非对角线为内积

右手定则 *Right-Hand Rule* —— 判断三维旋转正方向的经验法则

格拉姆矩阵性质 *Gram Matrix Property* —— 正交矩阵的格拉姆矩阵恒为单位矩阵

延伸阅读

回顾 · 8.2 缩放

本册前一小节 对角矩阵与缩放变换的细节展开

延伸阅读 · 8.5 正交投影

本册后续小节 正交投影、施密特正交化与QR分解

延伸阅读 · 8.6 镜像

本册后续小节 镜像变换与正交矩阵的另一种情形

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 用Python矩阵乘法完成：给定点(3, 4)绕原点逆时针旋转90度，算出旋转后的坐标。

Q3 平面旋转，先旋转30度、再旋转60度，分别计算旋转矩阵，再计算复合几何变换矩阵。

Q5 给定points=[(1,0),(0,1),(-1,0),(0,-1),(0,0)]，逆时针旋转45°并输出新坐标。

Q7 写代码比较：先绕x1旋转90度再绕x2轴旋转90度，与先绕x2再绕x1，结果是否相同。

Q9 了解什么是单位四元数(Unit quaternion)，了解如何用它完成三维空间旋转。

Q2 证明二维旋转矩阵的逆矩阵是其转置矩阵。

Q4 编写Python函数rotate_point(x1, x2, theta)，返回平面绕原点旋转后的坐标。

Q6 用matplotlib绘制一个单位正方形，绕原点逆时针旋转60°后再绘制旋转后的正方形。

Q8 了解什么是罗德里格旋转(Rodrigues' rotation)。

习题 Q10 – Q17

Q10 了解万向锁问题(Gimbal Lock)。

Q12 写一个Python函数`is_rotation(R)`，判断给定 2×2 矩阵是否为旋转矩阵。

Q14 构造 $R(30^\circ)$ 与缩放矩阵 $S = \text{diag}(2, 0.5)$ ，用Python比较 RS 与 SR 是否相等。

Q16 验证： $R(\theta)$ 与 $R(-\theta)$ 相乘结果为单位矩阵，并用可视化展示旋转后再旋转回原位。

Q11 了解复数，并了解复数和旋转有什么联系。

Q13 用numpy验证：任意角度 θ ， $R(\theta)$ 的行列式恒为1，且 $R(\theta) @ R(\theta).T$ 等于单位矩阵。

Q15 写Python函数`compose_rotations(*thetas)`，返回多个平面旋转角度复合后的总旋转矩阵。

Q17 对表1中30度、60度旋转矩阵，计算格拉姆矩阵 $R^T R$ ，验证结果是否为单位矩阵。

习题 Q18 – Q25

Q18 写Python函数`rotate_x1(x1,x2,x3,theta)`，返回三维点绕x1轴旋转后的坐标。

Q20 验证：绕同一轴的两次旋转 $R1(\theta_a)$ 与 $R1(\theta_b)$ 相乘满足交换律，绕不同轴则不满足。

Q22 扩展代码1的`plot_cube()`函数，添加参数控制立方体的透明度和边框颜色。

Q24 综合练习：构造 $R1(30^\circ)$ 、 $R2(45^\circ)$ 、 $R3(60^\circ)$ ，依次复合，用代码可视化最终旋转效果。

Q19 用numpy验证： $R1$ 、 $R2$ 分别绕x1、x2轴旋转90度，比较 $R1 @ R2$ 与 $R2 @ R1$ 是否相同。

Q21 用拉普拉斯展开手动计算 $R2$ 、 $R3$ 的行列式，验证结果均为1。

Q23 写一个Python函数`rotation_matrix_3d(axis, theta)`，根据axis('x1'/'x2'/'x3')返回对应旋转矩阵。

Q25 预习思考：下一节将介绍正交投影，行列式为0；对比旋转(行列式为1)，两者对基底有何不同影响？

10个可以和AI互动的Prompt

1 请解释为什么旋转矩阵的逆矩阵等于其转置矩阵，这与正交矩阵的定义有什么关系？

2 请证明：连续两次绕原点旋转满足交换律，即 $R(\alpha)R(\beta)=R(\beta)R(\alpha)=R(\alpha+\beta)$ 。

3 请用Python演示：先缩放再旋转与先旋转再缩放，为什么会得到不同结果？

4 请解释格拉姆矩阵 $R^T R = I$ 说明了旋转矩阵列向量的哪些几何性质。

5 请解释为什么二维旋转矩阵的行列式恒为1，这对图形的面积有什么影响？

10个可以和AI互动的Prompt

6 请解释为什么三维空间中绕不同轴的旋转不满足交换律，而绕同一轴的旋转可以交换。

7 请用Python验证：三维旋转矩阵 R_1 、 R_2 、 R_3 都是正交矩阵，行列式均为1。

8 请解释右手定则如何判断三维旋转的正方向，并用具体例子说明。

9 请解释罗德里格旋转公式与本节旋转矩阵之间的联系，为什么它更适合绕任意轴旋转？

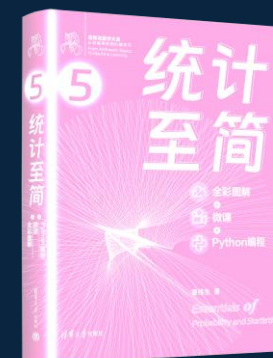
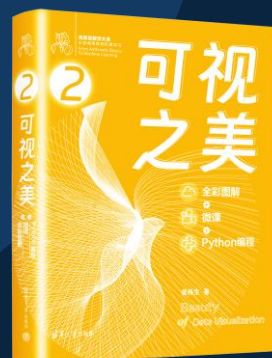
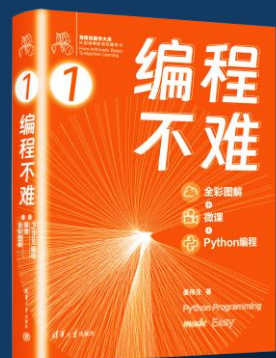
10 请解释单位四元数为什么能避免万向锁问题，相比旋转矩阵有什么优势？

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

